Notes of parallelism and concurrency in software

Dario Marcone

Jeudi 19 février 2026

Contents

1 Metrics **2**
1.1 Iron Law . . . 2
1.2 Amdahl's law . . . 3
1.3 Different type of mean . . . 3

2 Parallel computing **4**
2.1 Expressing parallelism . . . 4
2.2 Principles of parallel computing . . . 5

3 Hardware cache coherence **7**
3.1 Bus-based systems . . . 7
3.2 Point-to-point systems . . . 8

4 Optimizing software **10**
4.1 Coherence misses . . . 10
4.2 Work distribution and scheduling . . . 13

5 SIMD and vector **13**
5.1 AVX instructions . . . 14

6 Message passing **16**
6.1 MPI . . . 17

1 Metrics

1.1 Iron Law

$$\text{Processor Performance} = \frac{\text{Instructions}}{\text{Program}} \cdot \frac{\text{Cycles}}{\text{Instruction}} \cdot \frac{\text{Seconds}}{\text{Cycle}}.$$

-

- $\mathbf{\frac{Instructions}{Program}}$:

  The $\frac{\text{Instructions}}{\text{Program}}$ part represent the number of lines of code of your program, it is influenced by:

  - The compiler: depends of how many machine instructions per line of code.
  - The runtime system: depends of the language interpreted, the runtime overhead, etc

  > *Remark* The runtime system is a program given with the programmation language (ex. JVM for Java), that gives necessary services and mecanisms to the program during its execution, like memory alllocation, etc...

- $\mathbf{\frac{Cycles}{Instruction}}$:

  The $\frac{\text{Cycles}}{\text{Instruction}}$ part represent the number of cycles by instructions, it is influenced by:

  - The front-end cycles (instruction cache, branch predictor, etc...),
  - The back-end cycles (data cache, ALU, FPU, etc...),
  - The memory (latency, bandwith).

  > *Remark* Front-end part of the pipeline CPU is the instruction fetching, decoding and branch prediction part, and the banc-end is the computation, memory access and data stockage part.

- $\mathbf{\frac{Seconds}{Cycle}}$:

  The $\frac{\text{Seconds}}{\text{Cycle}}$ part is the clock frequency. We have to be careful! The cycles per instruction (CPI) is a good metric for uniprocessor, but not necessarly for multi processor. In multiprocessors, some core may have to wait for others, to have a specific information or whatever, so when the core wait, it will just do some spinning threads which is very fast $\implies$ a low CPI! But in fact, you dont execute the program particularely faster.
  It is better to use some metric for " useful work", like:

  - Transactions complete per second,
  - Request services per second.

Vocabulary parenthesis:

- **Latency:** time it takes for an event to complete
- **Bandwidth:** how many events per unit time

Reducing latency increases bandwidth but not vice versa.

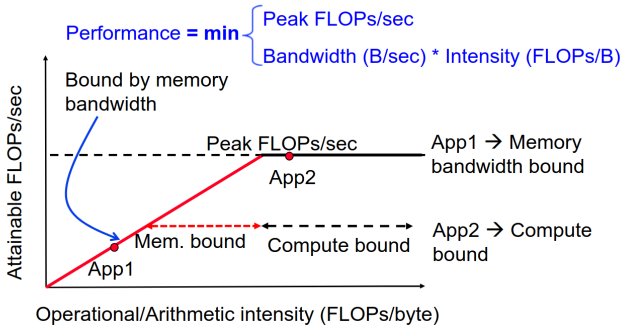
1.2 Amdahl's law

The amdahl's law shows that we have to improve efficency of what take the most of the time in our code execution.

$$\text{Speedup} = \frac{1}{\frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} + (1 - \text{Fraction}_{\text{enhanced}})}.$$

-

Roofline Model



- The x-axis correspond to the number of data used for computation: if it is low, then it means you load/write a lot of data but you don't make computation with them. You are limited by the speed of loading/writing of/in the memory, not by the CPU calculation speed $\implies$ memory bound.
- The y-axis correspond to the number of FLOPs that you can reach on the CPU, if you are doing a lot of computation by byte, then the CPU reach its maximum calculation speed $\implies$ peak FLOPs/sec, independent of memory.
- We can know by what we are limited by taking the minimum of

  $$\text{Performance} = \begin{cases} \text{Peak FLOPs/sec} \\ \text{Bandwidth} \cdot \text{Intensity} \end{cases}$$

  It means that

  if $B \cdot \text{Intensity} < \text{Peak FLOPs/sec} \implies$ we are limited by memory,

  if $B \cdot \text{Intensity} \geq \text{Peak FLOPs/sec} \implies$ we are limited by CPU.

Parallel efficiency

$$\text{Parallel efficiency} = \frac{\text{Speedup}}{\#\text{ of cores}}.$$

-

Vocabulary parenthesis:

- **Speedup(n)**: performance improvement of n cores over 1 core,
- **Costup(n)**: cost increase of n cores over 1 core.

1.3 Different type of mean

- Arithmetic mean:

$$\frac{1}{n}\sum_{i=1}^{n}\text{Time}_i,$$

- Harmonic mean:

$$\frac{n}{\sum_{i=1}^{n}\frac{1}{\text{Rate}_i}},$$

- Geometric mean:

$$\text{Relative Rate} = \frac{\text{Rate}}{\text{Rate}_{\text{ref}}} = \frac{\text{Time}_{\text{ref}}}{\text{Time}},$$

- Geometric mean:

$$\sqrt[n]{\prod_{i=1}^{n}\text{Relative Rate}_i} = \frac{\sqrt[n]{\prod_{i=1}^{n}\text{Rate}_i}}{\text{Rate}_{\text{ref}}}.$$

2 Parallel computing

2.1 Expressing parallelism

There was different era of core computing, first the **simple core era**, we were executing one instruction per cycle.
Then in **pre-multicore era**, we had one BIG fancy core:

- A lot of budget on run a single instruction very fast,
- More transitors, so faster branch predictor, larger cache hierarchy, etc...

Finally, we are in the **multicore era**, more transitors, but in simple cores: each core is slower (around 25%), but there is (for example) two cores $\rightarrow 2 \cdot 0.75 = 1.5$.

But now, we have to parallelize our code, either it will run on only one of the cores. There is different way expressing parallelism as programmer:

- Use existing parallel code through libraries: Spiral, ScalAPACK,
- Writing in a parallel programming language: HPF, CAF, Java, Scala,
- Writing in compiler directives: OpenMP, Intel TBB,
- Writing using a threading library: MPI, PVM, Pthreads,
- Domain Specific Languages (DSLs): Spiral, CUDA.

PThreads:

```
void parallel_cbrt(float* v, float* result, int N) {
    pthread_t thread_id;
    my_args args;
    args.N = N/2;
    args.v = v;
    args.result = result;
    pthread_create(&thread_id, NULL, my_thread_start, &args)
    my_cbrt(v + args.N, result + args.N, N - args.N);
    pthread_join(thread_id, NULL);
}
```

OpenMP: We can parallelize simply when loop iterations are independent of each other.

```
void my_cbrt(float *v, float *result, int N) {
    #pragma omp parallel for
    for (int i = 0; i < N; ++i) {
    float x = 1.f;
    for (int j = 0; j < 10; ++j)
    x = (1.f / 3.f) * (2.f*x + v[i] / (x*x));
    result[i] = x;
    }
}
```

Another idea of **multicore era**, is the **SIMD processing**: Single Instruction, Multiple Data. We add ALUs to increase our compute capability, the idea is to transform a loop where there is an instruction repeated on a lot of different data into an single instruction on a vector of a lot of data, then for a simple for loop:

```
for (int i = 0; i < 8; i++)
c[i] = a[i] + b[i];
```

We can compute this with one single operation of vector with the code:

```
__m256 va = _mm256_load_ps(a);
__m256 vb = _mm256_load_ps(b);
__m256 vc = _mm256_add_ps(va, vb);
```

`__m256` stand for the float type, the `_mm256` is the size of the vector, here 256 bits (8 float) and the load/add is the operation of the funcion.
GPUs are big SIMD processor!

2.2 Principles of parallel computing

- Finding enough paralellism,
- Division of work,
- Scaling,
- Communication and synchronization.

Finding enough paralellism: this is all the Amdahl's law part.

Division of work:

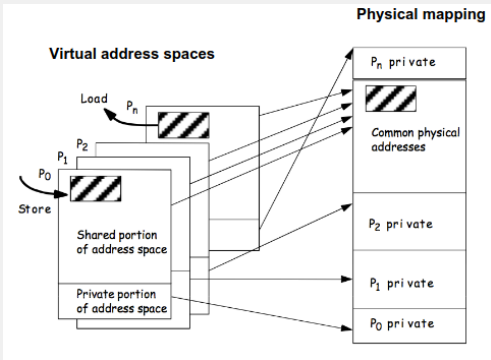
- Balance workload, give each parallel task the same rough amount of work.
- Reduce communication, computation is useful work, communication is overhead.
- Granularity, this is the ratio of computation to communication.

- Fine-grained parallelism: the idea is to split the work into very small tasks. There is not a lot of computation by thread but a lot of computation $\implies$ low granularity, but we need a fast communication favors: e.g. SIMD, GPU, ...
- Coarse-grained parallelism: the idea is to split the work into big tasks. We have less cache and TLB misses and less communication, we can tolerate, but its harder to divide the work into equal time part.

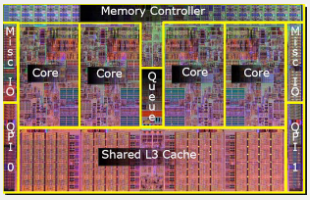
Communication & synchronization: We will see three different model of communication/synchronization:

- Message passing: the synchronization is at the I/O level, hadoop (e.g. datacenter).
  - There is complelety different address spaces, threads communicate via sending/receiving message, the synchronization is implicit in messages.
  - This is harder to program than shared memory, but more scalable.
  - Message passing is the most used communication system in datacenters.
- Shared memory: the synchronization is at the memory level, Scala (e.g. CPU).
  - Threads communicate by reading/writing to shared vars, that are like a big bulletin board (panneau d'affichage).
  - Any thread can read or write, communication is implicit in memory operations (no send/receive operations).
  - Non-uniform memory access (NUMA): all processors can access any memory location, but cost of memory access is different for different processors.
  - Implementation and some example:

> Each thread has its own virtual address, but shared parts maps to the same physical location.



> Any processor can directly reference any memory location (e.g. Intel Core i7 example (quad core)).



- Dataflow: the synchronization is at the processor level, TensorFlor (e.g. TPU).
  - In data-parallel computation, communication and synchronization are implicit, since each processor works on a different part of the vector and synchronizes (or joins) at the end.

3 Hardware cache coherence

A small reminder before starting: a cache contains usually the state (valid/invalid), the tag array and the data array that keeps cache blocks (a cache block == a cache line).
4Cs cache miss model:

- Compulsory: access data for the first time.
- Capacity: when the cache is not big enough.
- Conflict: when two addresses map to the same set and way of the cache evicting one of them.
- Coherence: when a block is removed due to coherence messages from another core.

Due to sharing data, incoherence appears between different caches. More specifically, when you're updating a value and reading a different copy. There is different ways to assure coherence we'll see two of them with their protocol.

3.1 Bus-based systems

In bus-based systems, we have to add independent "cache controllers" to each core where each controller: is a FSM, sees all remote processor actions, takes coherence actions depending on block's state and protocol.

MESI protocol: **M**odified, **E**xclusive, **S**hared and **I**nvalid states.

- Modified: this cache has updated the value and holds the authoritative copy.
- Exclusive: this cache has an exlusive copy.
- Shared: one or more caches hold the value.
- Invalid.

Core actions:

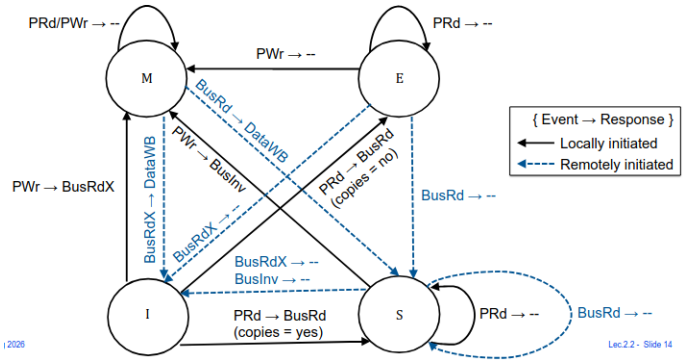
- PRd: processor attemps to read a line.
- PWr: processor attempts to write a line.

Remote actions:

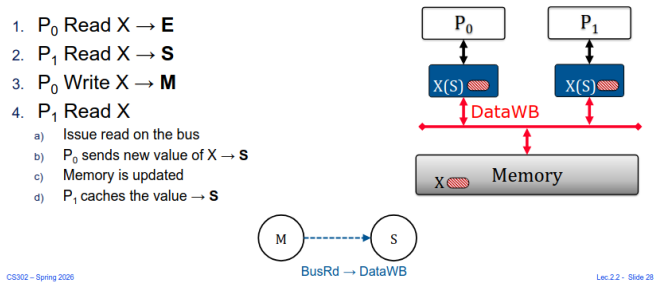
- BusRd: obtain copy of a line with no intent to modify.
- BusRdX: obtain copy of a line with intent to modify.
- BusInv: get rid of other copies.
- DataWB: put updated value of a line on the bus.

-

MESI protocol state diagram:



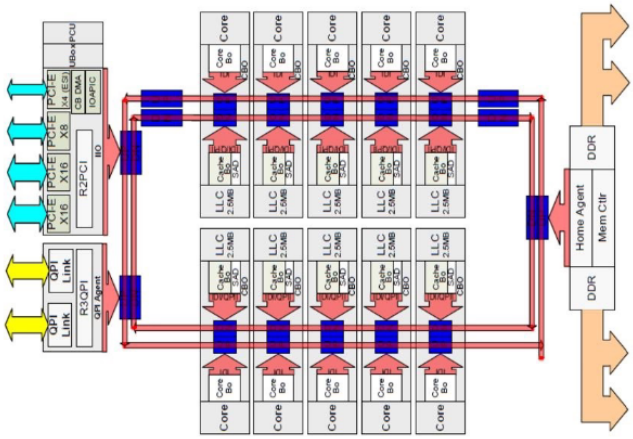
Example:



The main problem with bus-based systems is that the bus becomes a bottleneck: as the number of cores increases, the traffic on the bus grows, leading to contention and reduced performance. That's why we'll introduce the point-to-point network organization.

3.2 Point-to-point systems

In the point-to-point system, the message is more scalable and have an explicit source and destination. For example, Intel Skylake uses a ring:



> *Remark* In a ring (point-to-point) network, messages travel only along the path between sender and receiver, so links not on the path remain free, reducing overall traffic compared to a shared bus.

Now each core no longer sees all of the messages, so we need to add another protocol controller to the MESI protocol into the system.

Directory-based protocol:

In the directory-based protocole, we use a **coherence directory**. The directory tracks all caches that contain a particular block, then all messages leaving the core are first sent to the directory and it takes the appropriate actions, and respond to the original cache request. The directory serves as a per-block address or

Cache to directory messages:

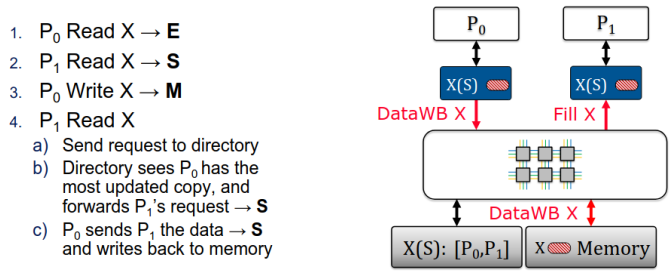
- GetRd: get copy to read,
- GetWr: get copy to modify,
- UpGrd: get rid of other copies,
- DataWB: write back a line,
- Ack: acknowledge message.

Directory to cache messages:

- DwnGrd: go from E/M →S,
- Inv: get rid of your copy,
- Fill: here is a copy of the line,
- Ack: acknowledge message.

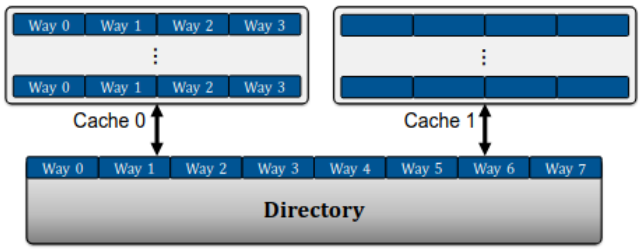
-

Example:



The **directory entry** contain: 2 bits storing the MESI coherence state, 39 bits containing the cache tag et $\log_2$ (Cores) bits identifying the core associated with this entry.

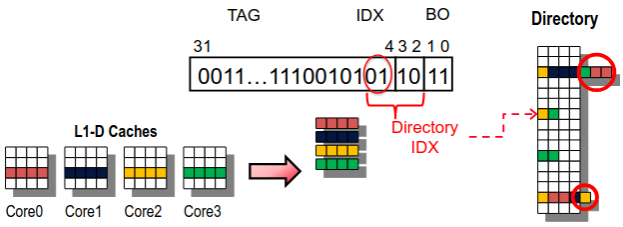
The size of directory is dependent of the number and the size of caches. If there is two processor with 4-ways associatives caches, then the directory must be 8-way associative:



But centralized directory can also be a source of contention, so we can distribute the directory across L2 cache banks, with each memory block having a home bank that tracks which L1 caches hold copies.

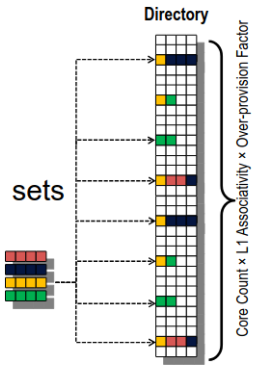
Access to the directory:

If we do the structure that we saw before for the directory, for four cores with 4-way caches, we need to have a 16-way associative cache! This is bad for latency and power. To allow this is the **sparse directory**, we can split the cache and take more tag bits to index.



Now the problem is that we can have conflit easily! The solution is to make a bigger directory and use more tag bits to index.

With this structre, we'll have low conflict rate, but a large area overhead...



4 Optimizing software

4.1 Coherence misses

Coherence adds extra misses when two cores acess the same cache line, then there is two types of coherence misses:

- **True sharing**: the producer/consumer communication when processors read and write the same variable.
- **False sharing**: processors updating different data which are placed in the same cache block.

True sharing, histogram example:

We have an input text and we want to count the occurences of each different characters in the text. We'll use OpenMP to divide the work across threads.

```
int histogram[N_BUCKETS];
int in_data[INPUT_SIZE];

#pragma omp parallel for shared (in_data, histogram)
for(int i = 0; i < INPUT_SIZE;i++) {
    unsigned bucket = calc_hist(in_data[i]);

    #pragma omp critical
    histogram[bucket]++;
}
```

Then, there is true sharing when processors modify the histogram `histogram['a']++;`. To avoid this, we have to preserve correct value. then we can

```
int histogram[N_BUCKETS];
int tmp_hist[N_THREADS][N_BUCKETS];
int in_data[INPUT_SIZE];

// Repartition loop
#pragma omp parallel for shared (in_data, tmp_hist)
for(int i = 0; i < INPUT_SIZE;i++) {
    int tid = omp_get_thread_num();
    unsigned bucket = calc_hist(in_data[i]);
    tmp_hist[tid][bucket]++;
}

// Merge loop
#pragma omp parallel shared (tmp_hist, histogram)
for(int i = 0; i < N_BUCKETS; i++) {
    int tid = omp_get_thread_num();
    #pragma omp critical
    histogram[i] += tmp_hist[tid][i];
}
```

Remark In the code, `shared (x, y)` tells OpenMP that all threads should see and access the same variable in memory (shared memory communication).

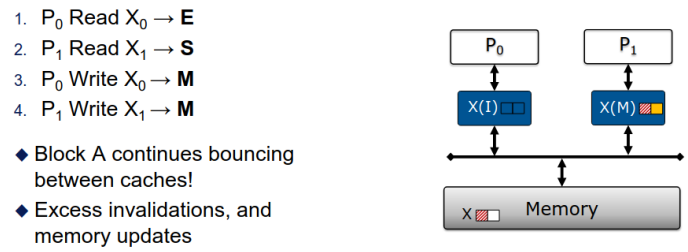
In the first code, we have `INPUT_SIZE` times the critical section and the read/write to the histogram. In the second one, we have `N_BUCKETS` times the number of threads, then if the size of the input is way bigger than the number of buckets times the number of threads, we eliminate a lot of critialt sections and true sharing.
Here is just an alternative of the merge loop that eliminate all critical sections, but every thread accesses a fraction of everyone's histogram.

```
// Merge loop
#pragma omp parallel for shared (tmp_hist, histogram)
for(int i = 0; i < N_BUCKETS; i++) {
    int tid;
    for (tid = 0; tid < omp_get_num_threads(); tid++) {
        histogram[i] += tmp_hist[tid][i];
    }
}
```

False sharing, odd numbers example:

```
int counters[NTHREADS];
int in_data[INPUT_SIZE];

#pragma omp parallel for shared (in_data,counters)
for(int i = 0; i < INPUT_SIZE;i++) {
    int tid = omp_get_thread_num();

    if(in_data[i] % 2) {
        counters[tid]++;
    }
}
```

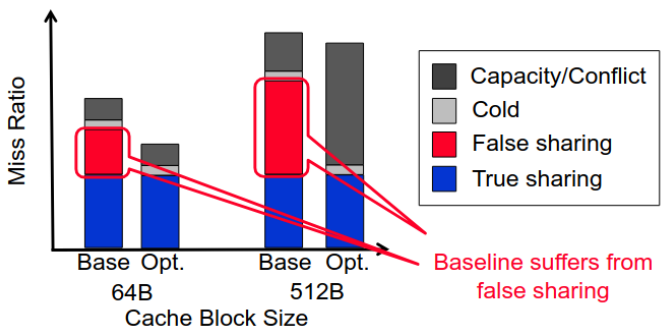
Nowadays, cache line are 64 byte in general, then it can store the four `int` on the same cache line. So everytime a core update its counter value, it invalidate every other cache line, this is false sharing!



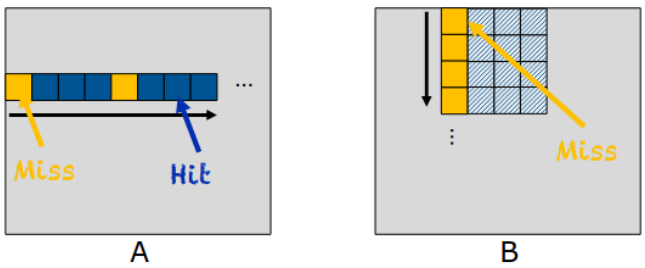
The solution here is to add padding, so the int are not on the same cache block anymore (e.g. 64 byte cache block line, a `int` is 4 byte, then we add a padding of 60 bytes).

```
typedef struct {
    unsigned _c;
    unsigned _padding[ BLK_SIZE/ UNSIG_SIZE - 1];
} PaddedCounter;
PaddedCounter counters[NTHREADS];

#pragma omp parallel for
{
    ...
    counters[tid]._c++;
}
```

We saw that eliminate false sharing take a lot of space, so the capacity and conflict misses increases. Then, the bigger the cache is, the less the padding is efficient to reduct global misses.

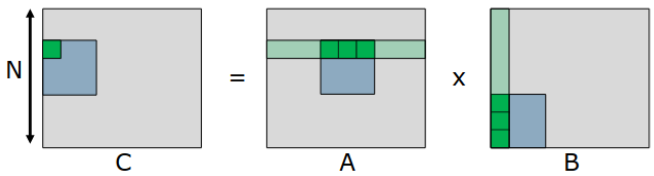


Block locality:
The locality of caches can be a problem in a program that, for example, multiply matrices, because we read the first matrice vertically and the second one horizontally.



Arrays are too large to fin in caches, then if we want to calculate $A \cdot B = C$, for every element of C if A and B are of size NxN, we'll have $1 + N\left[\frac{4}{\text{cache line size}}\right] + N$ misses! Then for the whole calculation, we obtain $\left(N^3(1+c) + N^2\right)$ which is $O\left(N^3\right)$ misses (where $c = \frac{\text{element size}}{\text{cache line size}}$).

To avoid this, we can do blocking. Then, for the whole array, we have to creat $\left(\frac{N}{n}\right)^2$ blocks and they have $2 \cdot \left[\frac{4 \cdot n}{\text{cache line size}}\right]$ misses, so finally we have $O\left(N^2\right)$ misses.



4.2 Work distribution and scheduling

OpenMP is a join/fork model where fork creates a eam of parallel threads and join synchronize and terminate threads at the end of parallel region. But fork and join comes with an overhead.

For a code like this, if the loop has too few iterations, the fork and join overhead is bigger than the savings made by the parallel execution. There is no need to parallelize the loop.

```
#pragma omp parallel for
for(i = 0; i < n; i++)
```

We can use the `if` clause to resolve this: `#pragma omp parallel for if(n > 5000)`.

5 SIMD and vector

We already introduce SIMD in the **section 2.1**. In the basic scalar pipelin, execution is in one simple ALU, and this code code

```
void daxpy(int n, double a, double* x, double* y){
    for(int i = 0; 0 < n; i++){
        y[i] = a * x[i] + y[i];
    }
}
```

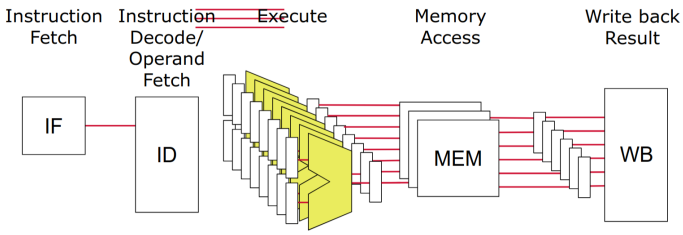
would look like this:

```
loop:
    lw r1, 0(r2) //load x[i]
    lw r6, 0(r3) //load y[i]
    mul r1, r1, r4  //a*x[i]
    add r6, r1, r6  //... + y[i]
    sw r6, 0(r3)  //store y[i]
    add r2, r2, 8
    add r3, r3, 8
    bne r2, r5, loop
```

If we are doing 1024 iterations, we'll have $8 \cdot 1024 = 8192$ instructions.
We can count the number of misses too, a size of double is 8B, a cache line 64B $\implies$ 8 elements per block. We have two loads by loop, then we know that we have $\frac{2 \cdot 1024}{8} = 256$ misses.

If we want to optimize this by having a out-of-order processor and a re-order buffer (c.f. computer architecture), knowing that a ROB is of size ≈ 500 instructions, it is only 6% of instructions that can be in the ROB. We can't augment the ROB size because it adds too much overhead $\implies$ out-of-order processors are not a very good solution.

Let's introduce vector pipeline:



The vector processors:

- Reduces instruction bloat,
- Simplifies dependency checking,
- Potentially reduce cache misses,
- Reduces control hazards.

-

Examples of vector instructions:

- `mul.v v1, v2, v1` does the dot product v1 · v2.
- `mul.sv v1, r2, v1` multiplies scalar r1 to all elements of v1.
- `lw.v v1, 0(r1)` loads vector v1 from address r1.
- `sw.v v1, 0(r1)` stores vector v1 at address r1.

Then our compiler code would look like this:

```
loop:
    lw.v v1, 0(r2) ; load x[i]
    lw.v v2, 0(r3) ; load y[i]
    mul.sv v1, r4, v1 ; a*x[i]
    add.v v1, v1, v2 ;     + y[i]
    sw.v v1, 0(r3) ; store y[i]
    add r2, r2, 512 ; 64*8
    add r3, r3, 512
    bne r2, r5, loop
```

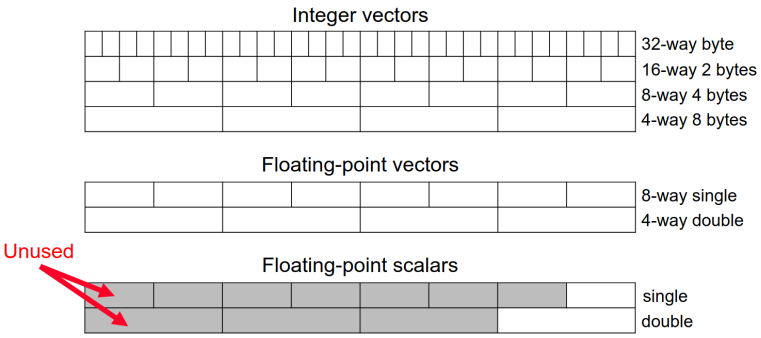
Now for 1024 iterations of our loop, we only have $8 \cdot \left(\frac{1024}{64}\right) = 128$ instructions. For misses, the vector load brings 64 elements at a time $\implies$ 8 cache lines $\implies 2 \cdot \left(\frac{1024}{64}\right) = 32$.

To apply SIMD we need a specific hardware:

- Vector registers to store multiple elements of a vector in a single register,
- Replicated functional units inside th epipeline of a core (multiple lane/ALUs),
- Support for decoding new instructions (SIMD instructions).

5.1 AVX instructions

AVX instructions are for 256-bit operations on x86 architecture.



Let's interpretate an AVX instructions e.g. `vaddpd`:

- `v` means that is an AVX instructions,
- `add` indicate the operation,
- `p` means packed, it can also be `s`, then it would be scalar (you can see upper),
- `s` stand for double-precision, can be `s` for single-precision, etc...

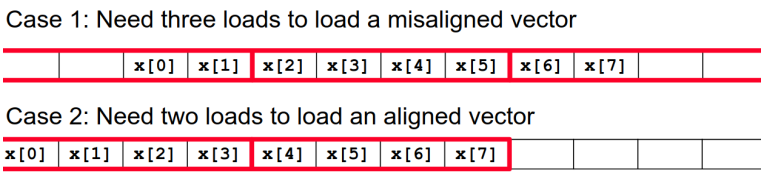
To use AVX instructions there is multiple way, the simplest is to auto-compile the code by using -O3 when compiling C code. But there is two main problems with the auto-compile solution. Aliasing and alignement.

Aliasing:
Let's take again our **daxpy** code, the compiler can't say at 100% that pointers are independent, then it won't vectorize. To solve this we have to say to the compiler that pointers don't overlap by changing the signature into

```
void daxpy(int n, double a, double* restrict x,
    double* restrict y);
```

Alignement:
Modern DRAM accesses are per four bytes and for better performances, accesses to vector elements should be aligned.

for example, to load an 8-element 32-bit vector:



To solve this, you can use `aligned_alloc` in C when allocating arrays and `_attribute_((aligned(32)))` for static arrays.

To summarize, the compiler cannot vectorize if:

- There is loop with unknown number of iterations at runtime,
- There is if-else statements or diverging condition inside loop,
- There is dependence between loop iterations,
- There is function calls in the loop body.

-

Another way to use AVX instructions is to optimize the code manually with "intrisics". Intrisics are compiler-provided functions that map directly to specific CPU instructions.

Different intrisics data types

Type	Meaning
__m256	8x single-precision floats
__m256d	4x double-precision floats
__m256i	32x 8-bit / 16x 16-bit / 8x 32-bit / 4x 64-bit integers
__m128	4x single-precision floats
__m128d	2x double-precision floats

Let's interpretate an AVX intrisics function e.g.
`_mm256_op_suffix(type param1, type param2, type param3)`:

- `mm256` means that we are working on the 256-bit registers,
- `op` is the operation to do, e.g. add, sub, load, etc...
- `suffix` is the type of the returned vector, e.g. ps, pd, epi32, epu64, etc...

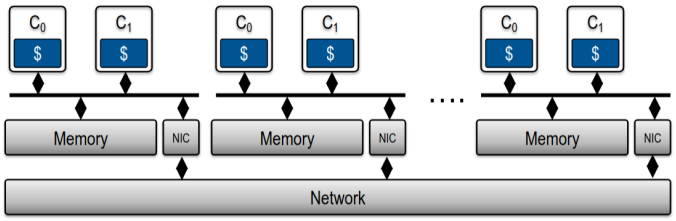
With this, our **daxpy** code would become

```
__m256d vecA = _mm256_set_pd(a, a, a, a);

for(int i = 0; i < n; i+=4) {
    __m256d vecX = _mm256_load_pd(&x[i]); //Load x[i]...
    __m256d vecY = _mm256_load_pd(&y[i]); //Load y[i]...
    __m256d vecZ = _mm256_mul_pd(vecA, vecX); //  a*x

    vecY = _mm256_add_pd(vecZ, vecY); // ... + y

    // Store result back to y
    _mm256_store_pd(&y[i], vecY);
}
```

AVX instrinsics C code optimize a lot normal code because cache misses are optimized too.

6 Message passing

Message passing is used when the memory is distributed, it happens and it's very useful for supercomputer or datacenters. So we have to use message passing when we have a particular hardware where: each node has its own memory, there is a network interface card (NIC) that connects nodes to a newtork.



Advantages of message passing are:

- The cost scalability: cache-coherent shared memory becomes expensive quite fast and distributed memory without cache coherence is common.
- The transparency: the communication is explicit (send/receive messages).
- The fault tolerence: if a process fail it doesn't crash the entire system.

Disavantages of message passing are:

- There is a higher communication overhead: syscalls, user-level library, etc ...
- It increases the code complexity.
- It is difficult to overlap with computation: while the message is travelling its hard to compute at the same time.

6.1 MPI

To apply message passing we use **MPI (Message Passing Interface)**. The model of MPI is a **SPMD (Single Program Multiple Data)** model: each process has a copy of the same program anf they all run at their own rate and can take different paths through the code.

A simple code using MPI looks like this:

```c
#include "mpi.h" //include the MPI libraries
#include <stdio.h>
int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv); //Initializes MPI env.
    printf("Hello, world!\n");
    MPI_Finalize(); //End of MPI section
    return 0;
}
```

MPI processes can be collected into groups and MPI identifies groups by their context.
MPI communicators are combination of a group and its context. Groups can sometimes overlap.

Some facts on communicators:
On starting an MPI program there is one predefined communicator called `MPI_COMM_WORLD` that contains the group of all processes.
A process is identified by a unique number within each communicator, its called the rank of a process, and the same process can have different rank in different communicators when groups overlap.
The utility of communicators is to minimize unnecessary communications, but simple programs usually only use `MPI_COMM_WORLD`.

If we want to customize our "Hello world":

```c
#include "mpi.h"
#include <stdio.h>
int main( int argc, char *argv[] )
{
    int rank, size;
    MPI_Init( &argc, &argv );
    //Get the rank of the process
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    //Get the # of the process
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    printf("I am process %d of %d.\n", rank, size);
    MPI_Finalize();
    return 0;
}
```

In MPI both process must take action to communicate: one have to send and the other have to receive.

Send: `MPI_Send` is a blocking functioin call, it returns when message has been delivered to the system but it doesn't imply that the message has been received by the target process. There exist variant with different effects:

- `MPI_Bsend`: it returns when the message is buffered in an application buffer (doesn't wait at all before returning),
- `MPI_Ssend`: it returns only when receiver has started to receive the message,
- `MPI_Rsend`: assuming receiver is ready (is waiting), message is sent asap.

Receive: `MPI_Recv` is also a blocking function call, its source can be `MPI_ANY_SOURCE` to accept data from any sender and the tag can be `MPI_ANY_TAG` to accept messages with any tag. The status contains further information but we can ignore with `MPI_STATUS_IGNORE`.

Communicating arrays:
There is different approach to send arrays with MPI, the first one would be to send elements one by one, or to send the entire array at once.

```
int A[1000];
for(int i = 0; i < 1000; i++) {
    MPI_Send(&A[i], 1, MPI_INT, rank, tag,
        MPI_COMM_WORLD);
}
```

Or,

```
int A[1000];
MPI_Send(A, 1000, MPI_INT, rank, tag, MPI_COMM_WORLD);
```

A hybrid approach is often the best (chunks of 100 elements), the first approach creats too much overhead and the second one solicit the memory and spam the cache if arrays are too big.
For short messages, latency dominates transfer time, but for long messages this is bandwith.

The **critical message size** is the amount of buffer that exists in the system, you can find it by computing

$$\text{latency} \cdot \text{bandwidth}.$$

Ping-pong between two processes example:

```
int ping_pong_count = 0;
int partner_rank = (rank + 1) % 2;
while (ping_pong_count < PING_PONG_LIMIT) {
    if (rank == ping_pong_count % 2) {
        // Increment the ping pong count before you
            send it
        ping_pong_count++;
        MPI_Send(&ping_pong_count, 1, MPI_INT,
            partner_rank, 0, MPI_COMM_WORLD);
        printf("%d sent and incremented %d to %d\n",
             rank,ping_pong_count, partner_rank);
    } else {
        MPI_Recv(&ping_pong_count, 1, MPI_INT,
            partner_rank, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
        printf("%d received %d from %d\n", rank,
            ping_pong_count,partner_rank);
    }
}
```

-

MPI functions are blocking functions, it means that sometimes deadlocks can occurs if the code isn't written correctly.

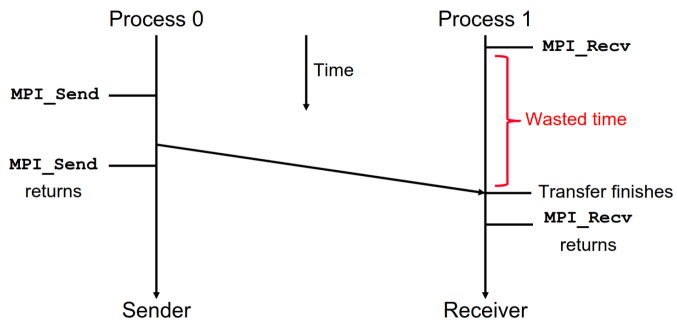
For example, in this code, the process 0 is stuck sending data, and process 1 is also stuck sending data. Neither can invoke `MPI_Recv` to the data.

```
if(rank == 0){
    MPI_Send(...);
    MPI_Recv(...);
} else if(rank == 1){
    MPI_Send(...);
    MPI_Recv(...);
}
```

Here, to fix this problem if order is known, we simply have to order functions correctly.

```
if(rank == 0){
    MPI_Send(...);
    MPI_Recv(...);
} else if(rank == 1){
    MPI_Recv(...);
    MPI_Send(...);
}
```

Due to blocking, we also have performance blocking:



To avoid this we can use **asynchronous communcation**. Non-blocking function calls return immediately after invocation but they doesn't provide any guarantee on the status of the message, so the programmer have to check manually for completion of message transfer. It requires more programmer effort but allows to overlap communication with computation and increase the efficiency and performance of the program.

Asynchronous transfer can be used in this situation. Process does not wait before moving on to different data.

```
*buf1 = 3;
MPI_Isend(buf1, 1, MPI_INT,..
*buf2 = 4;
```

But in this situation it's not possible. This is non-deterministinc, receiver can get either 3 or 4.

```
*buf1 = 3;
MPI_Isend(buf1, 1, MPI_INT,..
*buf1 = 4;
```

6.2 MPI collectives